

Import Libraries:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

Upload Dataset

```
from google.colab import files

uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving titanic.csv to titanic.csv

```
df = pd.read_csv("titanic.csv")
print(df.head())
```

```
   PassengerId  Survived  Pclass \
0             892         0       3
1             893         1       3
2             894         0       2
3             895         0       3
4             896         1       3
```

```
   Name                               Sex  Age  SibSp  Parch \
0  Kelly, Mr. James                    male  34.5     0     0
1  Wilkes, Mrs. James (Ellen Needs)  female  47.0     1     0
2  Myles, Mr. Thomas Francis        male  62.0     0     0
3  Wirz, Mr. Albert                  male  27.0     0     0
4  Hirvonen, Mrs. Alexander (Helga E Lindqvist) female  22.0     1     1
```

```
   Ticket    Fare  Cabin  Embarked
0  330911   7.8292   NaN         Q
1  363272  7.0000   NaN         S
2  240276   9.6875   NaN         Q
3  315154   8.6625   NaN         S
4  3101298 12.2875   NaN         S
```

▼ Descriptive statistics

```
#Numerical columns
print("Mean:")
print(df[["Age", "Fare"]].mean())

print("\nMedian:")
print(df[["Age", "Fare"]].median())

print("\nMode:")
print(df[["Age", "Fare"]].mode().iloc[0])

print("\nVariance:")
print(df[["Age", "Fare"]].var())

print("\nStandard Deviation:")
print(df[["Age", "Fare"]].std())
```

```
Mean:
Age      30.272590
Fare     35.627188
dtype: float64
```

```
Median:
Age      27.00000
```

```

Fare    14.4542
dtype: float64

Mode:
Age      21.00
Fare      7.75
Name: 0, dtype: float64

Variance:
Age      201.106695
Fare    3125.657074
dtype: float64

Standard Deviation:
Age      14.181209
Fare     55.907576
dtype: float64

```

✓ Pearson correlation

```

# Correlation between numerical variables
correlation = df[["Survived", "Pclass", "Age", "Fare"]].corr()

print(correlation)

# Correlation matrix visualization
plt.figure(figsize=(8, 6))

plt.imshow(correlation, cmap="coolwarm", interpolation="none")
plt.colorbar()

plt.xticks(range(len(correlation.columns)), correlation.columns)
plt.yticks(range(len(correlation.columns)), correlation.columns)

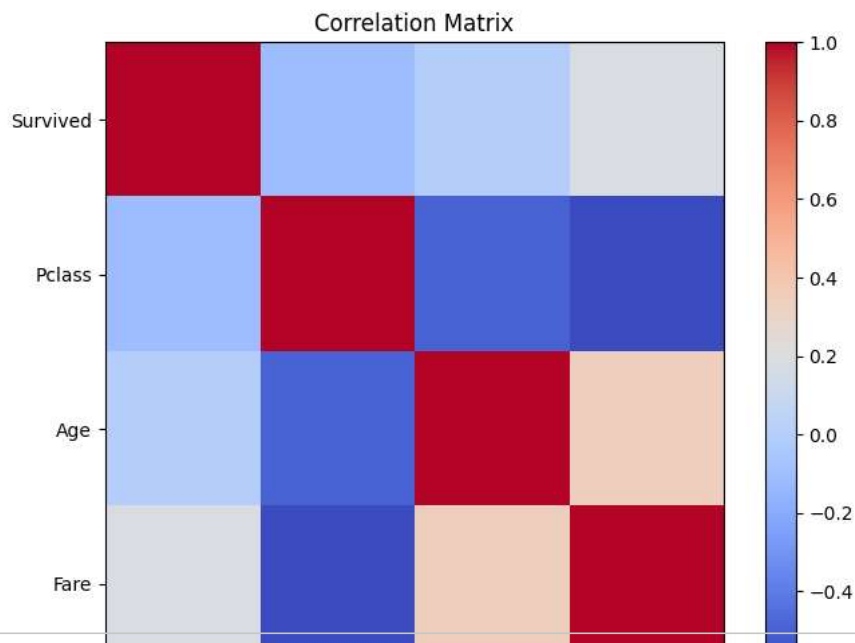
plt.title("Correlation Matrix")
plt.show()

```

```

Survived    Survived    Pclass    Age    Fare
Survived    1.000000   -0.108615  -0.000013  0.191514
Pclass      -0.108615    1.000000   -0.492143  -0.577147
Age         -0.000013  -0.492143    1.000000   0.337932
Fare         0.191514  -0.577147   0.337932    1.000000

```



```

# Remove missing Fare values
survived_fare = df[df["Survived"] == 1]["Fare"].dropna()
not_survived_fare = df[df["Survived"] == 0]["Fare"].dropna()

# Independent sample t-test
t_stat, p_value = stats.ttest_ind(
    survived_fare,
    not_survived_fare,
    equal_var=False
)

```

```
)

print("T-statistic:", t_stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject H0: There is a significant difference in Fare.")
else:
    print("Fail to reject H0: There is no significant difference in Fare.")

T-statistic: 3.4478682983060884
P-value: 0.0006845156351814127
Reject H0: There is a significant difference in Fare.
```

One-Way ANOVA

```
# Create groups based on passenger class
class1 = df[df["Pclass"] == 1]["Fare"].dropna()
class2 = df[df["Pclass"] == 2]["Fare"].dropna()
class3 = df[df["Pclass"] == 3]["Fare"].dropna()

# One-Way ANOVA
f_stat, p_value = stats.f_oneway(class1, class2, class3)

print("F-statistic:", f_stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject H0: There is a significant difference among passenger classes.")
else:
    print("Fail to reject H0: No significant difference among passenger classes.")

F-statistic: 129.89190776865902
P-value: 1.641767321164957e-44
Reject H0: There is a significant difference among passenger classes.
```

simple Linear Regression mode

```
# Select required columns
reg_data = df[["Age", "Fare"]].dropna()

# Independent variable
X = reg_data["Age"]

# Dependent variable
y = reg_data["Fare"]

# Add constant
X = sm.add_constant(X)

# Build regression model
model = sm.OLS(y, X).fit()

# Display results
print(model.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          Fare    R-squared:                0.114
Model:                  OLS    Adj. R-squared:            0.112
Method:                 Least Squares    F-statistic:          42.41
Date:                  Mon, 07 Sep 2026    Prob (F-statistic):    2.76e-10
Time:                  08:44:10    Log-Likelihood:       -1811.0
No. Observations:      331    AIC:                  3626.
Df Residuals:          329    BIC:                  3634.
Df Model:               1
Covariance Type:       nonrobust
=====
               coef    std err          t      P>|t|      [0.025     0.975]
-----
const         -3.2931     7.502     -0.439     0.661    -18.051     11.465
Age             1.4670     0.225     6.513     0.000     1.024     1.910
=====
Omnibus:                 241.179    Durbin-Watson:           2.017
Prob(Omnibus):            0.000    Jarque-Bera (JB):        2693.317
Skew:                     3.014    Prob(JB):                 0.00

```

Kurtosis: 15.607 Cond. No. 78.8
=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Regression Graph

```
plt.figure(figsize=(8, 5))

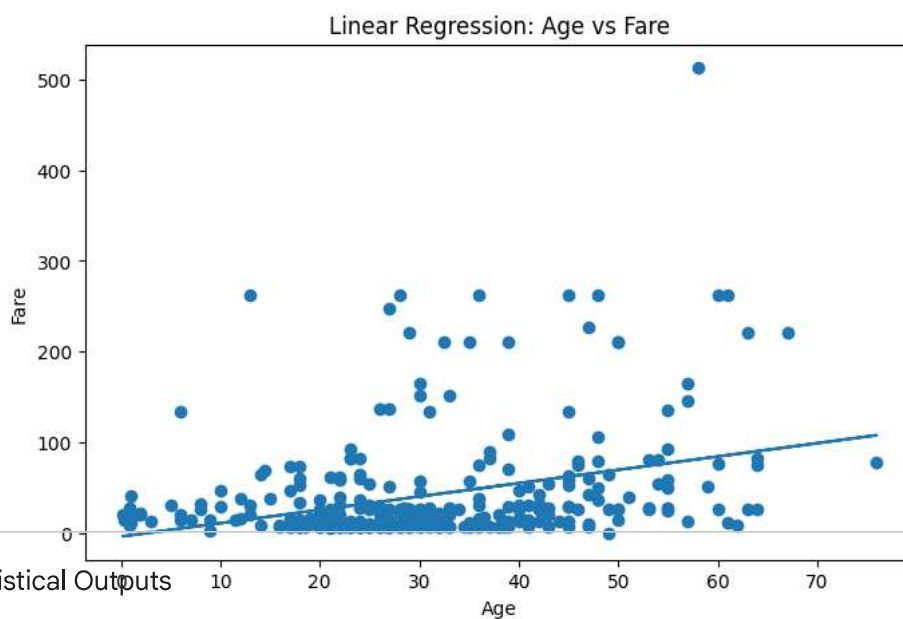
plt.scatter(reg_data["Age"], reg_data["Fare"])

# Prediction line
predicted = model.predict(X)

plt.plot(reg_data["Age"], predicted)

plt.xlabel("Age")
plt.ylabel("Fare")
plt.title("Linear Regression: Age vs Fare")

plt.show()
```



Statistical Outputs

```
print("Regression Coefficients:")
print(model.params)

print("\nP-values:")
print(model.pvalues)

print("\nConfidence Intervals:")
print(model.conf_int())

print("\nR-squared:")
print(model.rsquared)
```

```
Regression Coefficients:
const    -3.293097
Age       1.466976
dtype: float64

P-values:
const    6.609779e-01
Age      2.763372e-10
dtype: float64

Confidence Intervals:
           0         1
const -18.051091  11.464897
Age    1.023864   1.910087

R-squared:
0.11419775583782077
```

